AutoDD: Learning to Select Drift Detectors for Evolving Data Streams

Nilesh Verma [1], Albert Bifet [1,2], Bernhard Pfahringer [1], Maroua Bahri [3]
[1]*AI Institute, University of Waikato*, Hamilton, New Zealand
[2]*Télécom Paris, Institut Polytechnique de Paris*, Paris, France
[3]*CNRS, LIP6, Sorbonne Université*, Paris, France
{nilesh.verma, albert.bifet, bernhard.pfahringer}@waikato.ac.nz, maroua.bahri@lip6.fr

***Abstract*—Concept drift, the change in the distribution of streaming data over time, significantly impacts the performance of machine learning models. While numerous drift detection methods have been proposed, selecting and configuring the most suitable detector for a specific data stream remains a challenging task. Existing AutoML frameworks for streaming data that automate model selection rely on fixed drift detection strategies, leaving a critical gap in automation. This paper introduces *AutoDD* (Automated Drift Detector), a novel framework that jointly automates detector selection and hyperparameter configuration using a hierarchical learning approach. AutoDD extracts comprehensive online meta-features and employs a two-level hierarchical selection algorithm to adaptively choose both the detector and its configuration. Experiments on ten synthetic and real-world datasets show that AutoDD outperforms state-of-the-art detectors while maintaining computational efficiency.**

***Index Terms*—concept drift detection, algorithm selection, contextual bandits, online meta-learning, data streams**

I. Introduction

Concept drift, where the statistical properties of streaming data evolve over time, poses a significant challenge in data stream mining [1], [2], incremental learning [3], and non-stationary learning [4]. As data distributions change, machine learning models experience degraded performance, necessitating timely detection and adaptation. This phenomenon manifests in various forms, including sudden (abrupt) drift, gradual drift, incremental drift, and recurring drift. For instance, an electricity price prediction model may encounter sudden drift when market regulations change abruptly, experience gradual drift as consumer behaviour shifts over seasons, or observe recurring drift driven by daily demand cycles [4].

Over the past two decades, numerous drift detection methods have been developed to address this challenge [1], [4], spanning error rate-based [5]–[7], distribution-based [8], [9], uncertainty-based [10], and meta-learning [11] paradigms. Yet no single detector performs best across all conditions [1], [12], because each embeds assumptions about how drift manifests. As a result, a detector well suited to abrupt changes may respond slowly to gradual ones [4], [13], and the most appropriate choice can shift as the stream itself evolves [12]. However, a fundamental challenge remains unresolved in systematically selecting and configuring the most appropriate drift detector for a given data stream.

Automated Machine Learning (AutoML) has revolutionised the field by addressing similar selection challenges in batch learning scenarios [14]. Recent advancements have extended AutoML to streaming data, enabling automatic selection and configuration for supervised learning tasks [15], [16]. However, these systems treat drift detection as a fixed component, requiring manual specification and leaving a critical gap in the automation framework.

Automating this choice online is challenging because drift labels are unavailable as the stream arrives [1], [4], so detector quality must be inferred from a proxy reward derived from downstream model behaviour rather than from ground-truth change points [6], [11], and because the joint space of detectors and their configurations is too large to search as a single flat action set [14]. Unlike detector ensembles that run and combine many detectors at once [17], [18], or meta-learning methods that learn a fixed mapping offline [11], an effective solution must select and configure one detector online and continually refine this choice as the stream evolves. To bridge this gap, we introduce *AutoDD* (Automated Drift Detector), a modular framework that automates drift detector selection and configuration using a hierarchical learning approach, compatible with any model or AutoML pipeline. Figure 1 gives an overview of the framework. AutoDD makes three main contributions.

- We propose *AutoDD*, a novel framework for automated drift detector selection and configuration in evolving data streams.
- AutoDD formalises the problem of joint detector and hyperparameter selection and solves it via a two-level hierarchical bandit using online meta-features, LinUCB [19] for detector selection, and Thompson Sampling [20] for hyperparameter configuration.
- A comprehensive experimental study across ten benchmarks shows that AutoDD ranks first on mean accuracy and macro-F1 against state-of-the-art detectors, while maintaining computational efficiency.

The remainder of this paper is organised as follows. Section II reviews related work, Section III defines the problem, and Section IV presents AutoDD and its theoretical analysis. Sections V and VI describe the experimental setup and results, while Sections VII and VIII discuss limitations and conclude the paper.

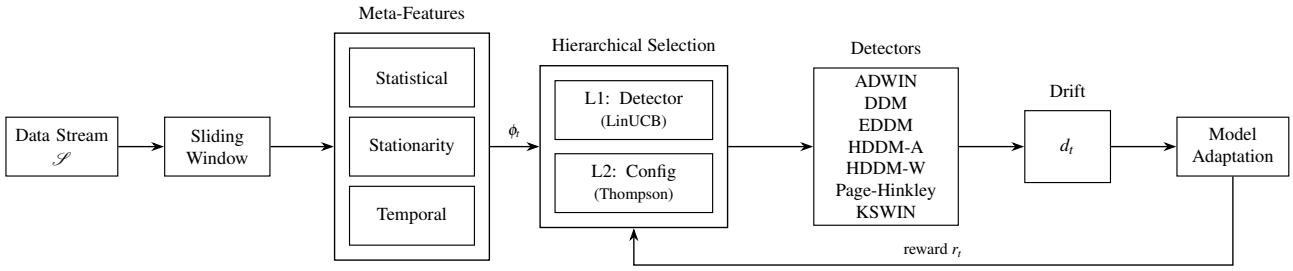


Fig. 1. Overview of the AutoDD framework. The incoming data stream $\mathcal{S}$ is buffered into a tumbling window, from which statistical, stationarity, and temporal meta-features form the context $\boldsymbol{\phi}_t$. Hierarchical selection then chooses a detector with LinUCB (Level 1) and a hyperparameter configuration with Thompson Sampling (Level 2) from the detector portfolio. The configured detector emits a drift signal d_t that triggers model adaptation, and the resulting reward r_t is fed back to update both selection levels.

II. Related Work

Drift detection methods can be broadly categorised into three paradigms. **Error rate-based methods** monitor fluctuations in classifier error rates, triggering alarms when changes are statistically significant. DDM [5] pioneered the warning-drift framework by monitoring prediction accuracy under binomial distribution assumptions, followed by EDDM [13] which focuses on distances between errors to detect gradual drift, HDDM [21] which applies Hoeffding's inequality with exponentially weighted moving averages, and ADWIN [6] which maintains adaptive sliding windows that automatically shrink when distribution changes are detected. Page-Hinkley [22] applies cumulative sum control charts to detect mean shifts in sequential data. Recent advances include MDDM [7], which uses McDiarmid's inequality for shorter detection delays with tighter concentration bounds, and ADDM [23], which models error rates as SETAR time series [24] to achieve lower false alarm rates. **Distribution-based methods** compare data distributions across windows by quantifying dissimilarity between new and historical data distributions [8], including approaches based on information theory [25], competence models [26], and the Kolmogorov-Smirnov test (KSWIN) [9] which detects changes in univariate feature distributions. KSWIN is a recent representative of the statistical two-sample test family, which also includes earlier detectors such as IKS [27] and WRS [17]. **Uncertainty and learning-based methods** address limitations of error-rate monitoring. PUDD [10] leverages prediction confidence for earlier detection by monitoring changes in prediction uncertainty distributions, while Meta-ADD [11] employs prototypical networks to classify drift types via meta-learning on diverse drift scenarios.

Recent developments in automated algorithm selection and hyperparameter optimisation (CASH) [14] have been extended to streaming scenarios [28]. However, these streaming AutoML systems automate supervised task selection while assuming a fixed, manually specified drift detector, so the detector itself is never selected or configured automatically. AutoDD addresses this gap by bringing the detector into the automation loop, learning online which detector and configuration best suit the current stream.

III. Problem Formulation

We consider a supervised data stream in which instances arrive sequentially and potentially without end. At each time step t, the stream produces an instance $(\mathbf{x}_t, y_t)$, where $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ is the feature vector and y_t is its label, drawn from a joint distribution $P_t(\mathbf{x}, y)$. The stream $\mathcal{S}$ exhibits concept drift when this distribution changes over time, that is, when

$$\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}, \qquad \exists t, t' : P_t(\mathbf{x}, y) \neq P_{t'}(\mathbf{x}, y). \tag{1}$$

Given a detector portfolio $\mathcal{D} = \{D_1, \ldots, D_K\}$ where each D_i has configuration space $\mathcal{H}_i$, AutoDD addresses the streaming detector selection and configuration problem. Inspired by the CASH formulation [14], we jointly optimise the detector choice and its hyperparameters:

$$(D_t^*, \boldsymbol{\eta}_t^*) = \arg\max_{D_i \in \mathcal{D}, \boldsymbol{\eta} \in \mathcal{H}_i} \mathbb{E}[\mathrm{Perf}(D_i(\boldsymbol{\eta}), \mathcal{S}_t) \mid \boldsymbol{\phi}_t] \tag{2}$$

where $\boldsymbol{\phi}_t$ is the stream context, $\mathcal{S}_t$ denotes the stream segment at time t, $\mathbb{E}[\cdot]$ is the expectation operator, and $\mathrm{Perf}(\cdot)$ denotes the downstream model performance.

IV. Automated Drift Detector

AutoDD employs an adaptive, closed-loop architecture to dynamically select and configure drift detectors. This architecture operates through several key steps. First, the system initialises the detector portfolio with standard hyperparameters and sets up the learning parameters. During online operation, meta-features characterising stream velocity, stationarity, and temporal patterns are continuously extracted from the incoming data stream. These meta-features serve as context for the LinUCB agent, which selects a detector from the portfolio. Thompson Sampling then chooses a hyperparameter configuration for the selected detector. The configured detector monitors the stream and triggers model adaptation upon detecting drift. Finally, performance feedback is computed based on detection outcomes, which updates both the LinUCB and Thompson Sampling components to refine future selections. Algorithm 1 formalises this adaptive learning cycle, which enables AutoDD to improve its detector selections over time.

Algorithm 1: AutoDD

Input: Stream $\mathcal{S}$, portfolio $\mathcal{D}$, window w, exploration α_{ucb}, regularisation λ_{reg}, configs per detector C, prior (a_0, b_0), reward constants $(\rho_{\text{tp}}, \kappa, \rho_{\text{fp}}, \rho_{\text{fn}}, \rho_{\text{tn}}, \tau_{\text{sig}})$

Output: Drift signals $\{d_t\}_{t=1}^{\infty}$

Initialise $\mathbf{A}_i \leftarrow \lambda_{\text{reg}}\mathbf{I}_m$, $\mathbf{b}_i \leftarrow \mathbf{0}$ for all $i \in [K]$
Initialise $a_{i,c} \leftarrow a_0$, $b_{i,c} \leftarrow b_0$ for all $i \in [K], c \in [C]$
Initialise model $\mathcal{M}_0$, tumbling window $W \leftarrow \emptyset$
foreach *instance* $(\mathbf{x}_t, y_t) \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{(\mathbf{x}_t, y_t)\}$
  if $|W| = w$ **then**
    $\boldsymbol{\phi}_t \leftarrow \textsc{Extract}(W)$ `// meta-features`
    for $i = 1$ **to** K **do**
      $\text{UCB}_i \leftarrow \boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha_{\text{ucb}} \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t}$
      // $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$
    $i_t \leftarrow \arg\max_i \text{UCB}_i$ `// select detector`
    for $c = 1$ **to** C **do**
      Sample $\psi_{i_t,c} \sim \text{Beta}(a_{i_t,c}, b_{i_t,c})$
    $c_t \leftarrow \arg\max_c \psi_{i_t,c}$ `// select configuration`
    $d_t \leftarrow D_{i_t}(\boldsymbol{\eta}_{i_t,c_t}).\textsc{Detect}(W)$
    if $d_t = 1$ **then**
      $\mathcal{M}_t \leftarrow \mathcal{M}_0$ `// reset model`
    Compute reward r_t using Eq. (10)
    $\mathbf{A}_{i_t} \leftarrow \mathbf{A}_{i_t} + \boldsymbol{\phi}_t\boldsymbol{\phi}_t^\top, \quad \mathbf{b}_{i_t} \leftarrow \mathbf{b}_{i_t} + r_t\boldsymbol{\phi}_t$
    $\tilde{r}_t \leftarrow (r_t + 1)/2$
    $a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t, \quad b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t)$
    $W \leftarrow \emptyset$

A. Algorithm Overview

AutoDD executes one selection cycle each time a tumbling window of w instances fills. Window-level quantities $(\boldsymbol{\phi}_t, i_t, c_t, d_t, r_t)$ are indexed by the window's closing instance t.

Meta-Feature Extraction: The meta-feature set comprises m features in three categories. *Statistical* features capture velocity, dimensionality, sparsity, noise level, and class imbalance. *Stationarity* features capture error volatility, KL divergence, autocorrelation, mean shift, and variance shift. *Temporal* features capture trend, periodicity, change frequency, change magnitude, and recurrence score. All features are computed incrementally over the window and normalised to $[0, 1]$ by running min-max normalisation, yielding the context vector $\boldsymbol{\phi}_t \in \mathbb{R}^m$.

Detector Portfolio: AutoDD maintains a diverse portfolio of K drift detectors spanning error-based (windowed, sequential, and Hoeffding-bound variants) and distribution-based paradigms. Table I summarises each detector with its hyperparameters, valid ranges, sampling scales, and default values from the literature. For each detector i, we maintain C hyperparameter configurations denoted $\boldsymbol{\eta}_{i,c}$ for $c \in [C]$, the first is always the literature default, while the remaining $C-1$ are sampled from the specified ranges using the appropriate logarithmic, linear, or integer scale.

TABLE I
DETECTOR PORTFOLIO WITH HYPERPARAMETER CONFIGURATIONS.

Detector	Type	Parameter	Range	Scale	Default
ADWIN	Error (window)	δ_{adwin}	[0.0001, 0.1]	log	0.002
Page-Hinkley	Error (CUSUM)	θ_{ph}	[10, 100]	int	50
		δ_{ph}	[0.001, 0.01]	log	0.005
DDM	Error-based	w_{thresh}	[1.5, 3.0]	linear	2.0
		d_{thresh}	[2.0, 4.0]	linear	3.0
EDDM	Error-based	α_{eddm}	[0.90, 0.99]	linear	0.95
		β_{eddm}	[0.85, 0.95]	linear	0.90
HDDM-A	Error-based	δ_{hddm}	[0.0001, 0.01]	log	0.001
HDDM-W	Error-based	λ_{hddm}	[0.01, 0.1]	log	0.05
KSWIN	Distribution-based	α_{ks}	[0.001, 0.05]	log	0.005
		w_{ks}	[50, 500]	int	100

B. Hierarchical Learning Approach

The two-level decomposition adapts detector choice and hyperparameter configuration jointly without treating all $K \times C$ detector-configuration pairs as a single flat action space.

Level 1 - Detector Selection via LinUCB: We formulate detector selection as a contextual multi-armed bandit problem where context is $\boldsymbol{\phi}_t$ (meta-features), arms are K detectors, and reward is detection performance. LinUCB [19] assumes linear reward:

$$r_{i,t} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* + \epsilon_{i,t} \tag{3}$$

where $\boldsymbol{\theta}_i^* \in \mathbb{R}^m$ is the unknown parameter vector for detector i. The algorithm selects the detector with the highest upper confidence bound:

$$i_t = \arg\max_{i \in [K]} \left(\boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha_{\text{ucb}} \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t} \right) \tag{4}$$

where $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$ is the ridge regression estimate, with

$$\mathbf{A}_i = \lambda_{\text{reg}}\mathbf{I}_m + \sum_{s:i_s=i} \boldsymbol{\phi}_s\boldsymbol{\phi}_s^\top, \qquad \mathbf{b}_i = \sum_{s:i_s=i} r_s\boldsymbol{\phi}_s. \tag{5}$$

Here, $\mathbf{A}_i \in \mathbb{R}^{m \times m}$ is the regularised design matrix with regularisation parameter $\lambda_{\text{reg}} > 0$, and $\mathbf{b}_i \in \mathbb{R}^m$ is the cumulative reward-weighted context vector for detector i. The parameter $\alpha_{\text{ucb}} > 0$ controls exploration-exploitation balance.

Level 2 - Hyperparameter Selection via Thompson Sampling: Given the selected detector D_{i_t}, we adaptively configure its hyperparameters using Thompson Sampling [20]. For each configuration $c \in [C] = \{1, \ldots, C\}$ of detector i, we maintain Beta distribution parameters $(a_{i,c}, b_{i,c})$ initialised to a uniform prior (a_0, b_0). At each window, we sample:

$$\psi_{i_t,c} \sim \text{Beta}(a_{i_t,c}, b_{i_t,c}) \quad \forall c \in [C] \tag{6}$$

and select configuration:

$$c_t = \arg\max_c \psi_{i_t,c} \tag{7}$$

After observing reward r_t, we update:

$$a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t \tag{8}$$

$$b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t) \tag{9}$$

where $\tilde{r}_t = (r_t + 1)/2$ normalises the reward to $[0, 1]$.

C. Evaluation and Adaptation

Without ground-truth drift labels, we use the downstream model's accuracy drop as a proxy reward:

$$r_t = \begin{cases} \rho_{\text{tp}} + \min(\kappa\,\Delta a_t, \rho_{\text{tp}}) & d_t = 1, \Delta a_t > \tau_{\text{sig}} \\ -\rho_{\text{fp}} & d_t = 1, \Delta a_t \le \tau_{\text{sig}} \\ -\rho_{\text{fn}} - \min(\Delta a_t, \rho_{\text{fn}}) & d_t = 0, \Delta a_t > \tau_{\text{sig}} \\ \rho_{\text{tn}} & d_t = 0, \Delta a_t \le \tau_{\text{sig}} \end{cases} \tag{10}$$

where $d_t \in \{0, 1\}$ is the window-level alarm, $\Delta a_t = a_{t-w} - a_t$ is the accuracy drop between consecutive windows, τ_{sig} is the significance threshold, $\rho_{\text{tp}}, \rho_{\text{fp}}, \rho_{\text{fn}}, \rho_{\text{tn}} > 0$ are the base rewards for the four outcomes, and κ scales the drop. The capped $\min(\cdot)$ terms keep $r_t \in [-1, 1]$, as the regret analysis requires. On an alarm the model resets to $\mathcal{M}_0$ and resumes incremental learning. The deployed implementation additionally updates Level 1 at each detected drift while updating Level 2 only at window boundaries.

D. Computational Complexity

All meta-features are linear-time window statistics except the FFT-based periodicity, so the per-window time and space complexity of AutoDD are

$$T_{\text{win}} = O(wd_{\text{in}} + w \log w + Km^2),\ S_{\text{win}} = O(Km^2 + wd_{\text{in}}), \tag{11}$$

where w is the AutoDD window size, d_{in} is the input feature dimensionality, K is the number of detectors, and m is the meta-feature count. Time is dominated by meta-feature extraction and LinUCB matrix operations, and space by the K covariance matrices and the window buffer.

E. Theory

We provide proxy-regret bounds for the two selection levels under idealised contextual-bandit assumptions.

Assumptions. We adopt standard assumptions from the contextual bandit literature [29], [30]:

- **(A1) Bounded features:** $\|\boldsymbol{\phi}_t\|_2 \le L$, with $L = \sqrt{m}$ under $[0, 1]$ normalisation.
- **(A2) Linear realizability:** For each detector i, there exists $\boldsymbol{\theta}_i^*$ with $\|\boldsymbol{\theta}_i^*\|_2 \le S$ such that

$$\mathbb{E}[r_{i,t} \mid \boldsymbol{\phi}_t] = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^*. \tag{12}$$

- **(A3) Sub-Gaussian noise:** The centred reward noise is conditionally σ_{noise}-sub-Gaussian, with $\sigma_{\text{noise}} = 1$ valid for rewards in $[-1, 1]$.
- **(A4) Proxy fidelity:** The reward in Eq. (10) is, in expectation, monotone in true detection quality under Eq. (2). A4 motivates the proxy but is not required by the bounds below.

Remark. Although P_t is non-stationary, A2 assumes that changes affecting expected reward are sufficiently represented by $\boldsymbol{\phi}_t$, rather than by time-varying $\boldsymbol{\theta}_i^*$.

Theorem 1 (Level 1 Proxy Regret). Fix each detector's configuration and assume A1–A3. With horizon T and confidence $\delta_{\text{conf}} \in (0, 1)$, use

$$\alpha_{\text{ucb}} = \sigma_{\text{noise}} \sqrt{m \log\left(1 + \frac{TL^2}{\lambda_{\text{reg}} m}\right) + 2 \log\left(\frac{K}{\delta_{\text{conf}}}\right)} + \sqrt{\lambda_{\text{reg}}} S. \tag{13}$$

Then disjoint-arm LinUCB achieves, with probability at least $1 - \delta_{\text{conf}}$,

$$R_1(T) = \textstyle\sum_{t=1}^{T} \left(\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^*\right) = \tilde{O}(m\sqrt{KT}). \tag{14}$$

Proof Sketch. Self-normalised confidence bounds [29] give

$$\|\boldsymbol{\theta}_i^* - \hat{\boldsymbol{\theta}}_i\|_{\mathbf{A}_i} \le \alpha_{\text{ucb}}. \tag{15}$$

Hence the UCB rule yields

$$\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \le 2\alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}}. \tag{16}$$

Applying the elliptical potential lemma [31] to the K estimators and Cauchy–Schwarz gives

$$R_1(T) = \tilde{O}(m\sqrt{KT}). \qquad \square$$

Proposition 2 (Level 2 Proxy Regret). Fix detector i and suppose its C configurations generate independent bounded rewards from stationary distributions. Let $\mu_{i,c}$ be the mean reward of configuration c, $\mu_i^* = \max_c \mu_{i,c}$, and $R_2(T) = \sum_t (\mu_{i_t}^* - \mu_{i_t,c_t})$. For idealised Beta–Bernoulli Thompson Sampling, with $N_i(T)$ selections of detector i,

$$\mathbb{E}[R_2(T)] = \sum_{i=1}^{K} O\left(\sqrt{C\, N_i(T) \log(1 + N_i(T))}\right). \tag{17}$$

Proof Sketch. Normalise rewards as $\tilde{r}_t = (r_t + 1)/2 \in [0, 1]$ and apply the standard Bernoulli reduction. Thompson Sampling then gives

$$O\left(\sqrt{C\, N_i(T) \log(1 + N_i(T))}\right) \tag{18}$$

regret for detector i [32]. Since $\sum_i N_i(T) = T$, Cauchy–Schwarz gives

$$\mathbb{E}[R_2(T)] = O\left(\sqrt{CKT \log(1 + T)}\right). \tag{19}$$

The fractional Beta update in Eq. (8) is therefore an implementation surrogate rather than being directly covered by this bound. $\square$

The two bounds apply to idealised sub-problems and do not compose into regret against a joint detector–configuration oracle. Treating them as separable gives the indicative rate

$$R_{\text{proxy}}(T) \lesssim \tilde{O}(m\sqrt{KT}) + O\left(\sqrt{CKT \log(1 + T)}\right), \tag{20}$$

which we report only as a heuristic surrogate.

TABLE II
DATASET CHARACTERISTICS (MULTIPLE* = SUDDEN, GRADUAL, INCREMENTAL, AND RECURRING DRIFT TYPES)

Dataset	Type	Instances	Features	Drift Type
SEA	Synthetic	500,000	3	Multiple*
HYP	Synthetic	500,000	10	Multiple*
AGR	Synthetic	100,000	9	Multiple*
RBF	Synthetic	200,000	10	Multiple*
RTG	Synthetic	100,000	30	Multiple*
Elec	Real-world	45,312	8	Unknown
Weather	Real-world	18,159	8	Unknown
Spam	Real-world	4,601	57	Unknown
Airlines	Real-world	539,383	7	Unknown
Poker	Real-world	829,201	10	Unknown

V. EXPERIMENTS

A. Datasets and Base Models

We evaluate AutoDD on ten benchmark datasets comprising five synthetic and five real-world streams, with characteristics detailed in Table II. To isolate drift detection performance from model selection effects, all methods use two fixed base models, GaussianNB and Hoeffding Adaptive Tree (HAT) [33]. These two learners represent fundamentally different model families (probabilistic and tree-based) and are the standard benchmark base learners used in the data stream mining literature [2], [17].

B. Baselines

We compare AutoDD against twelve baselines, namely a no-detector baseline and seven portfolio detectors (ADWIN, DDM, EDDM, HDDM-A, HDDM-W, Page-Hinkley, KSWIN). We also include four state-of-the-art methods, MDDM [7], ADDM [23], PUDD [10], and Meta-ADD [11]. All detectors use recommended hyperparameter settings from the literature. The portfolio includes one recent representative per detector family rather than every variant. Because AutoDD's selection mechanism is detector-agnostic, a larger portfolio extends only the action set, not the method.

C. Evaluation Protocol

We measure prequential accuracy as the percentage of correct predictions using test-then-train evaluation [2]. On the synthetic streams we additionally report prequential macro-averaged F1 over the class labels under the same protocol. We stress that this is the F1 of the downstream classifier, not a drift-detection F1 scored against ground-truth change points. The benchmark includes continuously drifting streams such as RTG, for which no discrete ground-truth drift positions exist, so a detection-matching score is undefined here and we do not report one. For real-world datasets, we report the number of detected drifts (DN) instead. The two are complementary. Accuracy is dominated by the majority class, whereas macro-F1 weights classes equally and so exposes the minority-class collapse that follows when a detector adapts too late or resets too often. Neither scores detection timing directly, which we treat as a limitation of proxy-based evaluation. Following the benchmarking guidance [12], we interpret DN relative to the consensus detection range across methods rather than as an absolute target, treating values far above or below this range as over- and under-detection respectively.

D. Implementation Details

AutoDD uses all m meta-features with a fixed window size, bandit parameters, and C configurations per detector throughout. Results are averaged over 10 runs with random seeds uniformly sampled from the range [1, 100]. Implementation uses Python with River [34] on a MacBook Pro with Apple M2 Pro and 16GB RAM.[1]

VI. RESULTS

We conduct a comprehensive evaluation of AutoDD against classical detectors and state-of-the-art methods.

A. Synthetic Datasets

Tables III and IV compare all methods on the five synthetic benchmarks. AutoDD achieves the best macro-F1 on every synthetic stream under both base models and the best mean accuracy under GaussianNB, with Meta-ADD ranking second across most datasets. The advantage is not uniform on accuracy. Under both base models, Meta-ADD is more accurate on RBF and RTG, the two highest-dimensional streams, though AutoDD retains the better macro-F1 on both. Under HAT these two streams give Meta-ADD the higher mean accuracy. The advantage is largest on datasets with abrupt drift, where adaptive detector switching is most beneficial, and narrows on gradual drift where Meta-ADD's learned representations are competitive. Fixed detectors exhibit inconsistent rankings across drift types, motivating the adaptive selection approach (Figures 2a–2b).

B. Real-world Datasets

Tables V and VI report results on real-world benchmarks where drift patterns are unknown. AutoDD ranks first on mean accuracy under both base models, with the improvement consistent across all five datasets. Detection counts remain within the consensus range of the compared methods, avoiding both extremes of over- and under-detection. On Elec, Weather, and Spam, AutoDD reports the same detection count as the strongest comparators, so its accuracy gain comes from when it adapts rather than from detecting more often. Figures 2c and 2d confirm that AutoDD tracks or leads the baselines on the Poker stream and recovers promptly after accuracy drops.

C. Statistical Significance

Figures 3a and 3b show Critical Difference diagrams from the Friedman–Nemenyi test ($\alpha = 0.05$). AutoDD achieves the best mean rank under both base models, and the No Detector baseline ranks last, confirming that drift detection is beneficial across these benchmarks. AutoDD is separated from every fixed portfolio detector by more than the critical difference, while Meta-ADD, PUDD, ADDM, and MDDM fall within

[1]Code and datasets are available at https://github.com/TechyNilesh/AutoDD

TABLE III

SYNTHETIC DATASETS UNDER GAUSSIANNB. RANK IN (·). **BOLD** = BEST, UNDERLINE = SECOND-BEST PER COLUMN.

Method	SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
No Detector	83.8 (11)	0.40 (10)	76.4 (11)	0.34 (8)	**78.8 (1)**	0.41 (10)	50.4 (13)	0.09 (10)	58.2 (11)	0.10 (7)	69.5 (12)	0.27 (10)
ADWIN	84.0 (10)	0.61 (9)	81.9 (9)	0.00 (9)	71.5 (10)	0.51 (7)	69.6 (10)	0.00 (11)	64.5 (9)	0.00 (8)	74.3 (10)	0.22 (13)
DDM	83.8 (11)	0.00 (12)	86.1 (7)	0.63 (7)	71.6 (9)	0.49 (9)	71.3 (4)	0.64 (4)	64.4 (10)	0.00 (8)	75.4 (7)	0.35 (9)
EDDM	84.3 (9)	0.40 (10)	85.7 (8)	0.63 (7)	65.3 (11)	0.24 (11)	67.7 (12)	0.38 (9)	65.0 (8)	0.40 (5)	73.6 (11)	0.41 (8)
HDDM-A	84.6 (7)	0.69 (6)	81.5 (10)	0.00 (9)	72.2 (8)	0.63 (6)	69.7 (9)	0.00 (11)	64.4 (10)	0.00 (8)	74.5 (9)	0.26 (11)
HDDM-W	84.4 (8)	0.66 (7)	86.6 (6)	0.71 (4)	73.1 (6)	0.67 (5)	70.1 (6)	0.51 (8)	65.8 (6)	0.56 (4)	76.0 (6)	0.62 (6)
Page-Hinkley	84.7 (6)	0.70 (5)	81.5 (10)	0.00 (9)	71.5 (10)	0.49 (9)	69.4 (11)	0.00 (11)	64.5 (9)	0.00 (8)	74.3 (10)	0.24 (12)
KSWIN	83.7 (12)	0.35 (11)	85.7 (8)	0.65 (6)	71.5 (10)	0.50 (8)	70.0 (7)	0.61 (6)	65.2 (7)	0.39 (6)	75.2 (8)	0.50 (7)
MDDM	86.7 (4)	0.65 (8)	87.2 (5)	0.68 (5)	72.3 (7)	0.71 (4)	69.8 (8)	0.56 (7)	66.1 (5)	0.56 (4)	76.4 (5)	0.63 (5)
ADDM	87.7 (3)	0.83 (3)	88.7 (4)	0.73 (3)	73.6 (5)	0.67 (5)	70.9 (5)	0.62 (5)	67.6 (4)	0.71 (3)	77.7 (4)	0.71 (4)
PUDD	86.2 (5)	0.81 (4)	89.5 (3)	0.84 (2)	73.7 (4)	0.72 (3)	73.4 (3)	0.72 (2)	67.8 (3)	0.71 (3)	78.1 (3)	0.76 (3)
Meta-ADD	88.9 (2)	0.87 (2)	90.8 (2)	**0.91 (1)**	74.1 (3)	0.79 (2)	**78.0 (1)**	0.70 (3)	**72.9 (1)**	0.76 (2)	80.9 (2)	0.81 (2)
AutoDD	**93.1 (1)**	**0.90 (1)**	**94.6 (1)**	**0.91 (1)**	78.5 (2)	**0.84 (1)**	74.3 (2)	**0.81 (1)**	69.2 (2)	**0.84 (1)**	**81.9 (1)**	**0.86 (1)**

TABLE IV

SYNTHETIC DATASETS UNDER HAT. RANK IN (·). **BOLD** = BEST, UNDERLINE = SECOND-BEST PER COLUMN.

Method	SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
No Detector	86.2 (10)	0.42 (9)	79.1 (13)	0.36 (10)	81.3 (2)	0.43 (11)	53.1 (12)	0.11 (8)	60.8 (13)	0.12 (10)	72.1 (13)	0.29 (11)
ADWIN	88.1 (8)	0.64 (7)	85.5 (12)	0.06 (11)	74.6 (10)	0.55 (8)	73.3 (10)	0.07 (9)	68.3 (11)	0.11 (11)	78.0 (11)	0.29 (11)
DDM	88.0 (9)	0.10 (10)	89.0 (7)	0.67 (8)	74.7 (9)	0.52 (10)	74.6 (5)	0.69 (3)	68.1 (12)	0.13 (9)	78.9 (7)	0.42 (9)
EDDM	88.0 (9)	0.46 (8)	88.6 (8)	0.68 (7)	69.2 (12)	0.29 (12)	71.8 (11)	0.42 (7)	68.8 (9)	0.47 (7)	77.3 (12)	0.46 (8)
HDDM-A	88.3 (6)	0.64 (7)	86.8 (10)	0.06 (11)	75.1 (7)	0.63 (7)	73.8 (7)	0.07 (9)	69.1 (7)	0.11 (11)	78.6 (9)	0.30 (10)
HDDM-W	88.4 (5)	0.65 (6)	89.5 (6)	0.72 (6)	75.9 (5)	0.67 (6)	73.9 (6)	0.53 (6)	69.7 (6)	0.58 (6)	79.5 (6)	0.63 (6)
Page-Hinkley	88.2 (7)	0.65 (6)	86.0 (11)	0.06 (11)	74.8 (8)	0.52 (10)	73.5 (9)	0.07 (9)	68.5 (10)	0.11 (11)	78.2 (10)	0.28 (12)
KSWIN	88.0 (9)	0.42 (9)	88.3 (9)	0.66 (9)	74.5 (11)	0.53 (9)	73.7 (8)	0.63 (4)	68.9 (8)	0.43 (8)	78.7 (8)	0.53 (7)
MDDM	89.1 (4)	0.67 (5)	90.3 (5)	0.75 (5)	75.5 (6)	0.75 (4)	73.8 (7)	0.60 (5)	69.8 (5)	0.61 (5)	79.7 (5)	0.68 (5)
ADDM	89.8 (3)	0.86 (3)	91.9 (4)	0.77 (4)	76.6 (4)	0.72 (5)	75.0 (4)	0.69 (3)	71.3 (3)	0.77 (3)	80.9 (4)	0.76 (4)
PUDD	89.8 (3)	0.83 (4)	92.6 (2)	0.83 (3)	77.1 (3)	0.76 (3)	76.4 (2)	0.74 (2)	71.9 (2)	0.74 (4)	81.6 (3)	0.78 (3)
Meta-ADD	90.9 (2)	0.91 (2)	92.5 (3)	0.92 (2)	77.1 (3)	0.81 (2)	**84.8 (1)**	0.74 (2)	**80.4 (1)**	0.79 (2)	**85.1 (1)**	0.83 (2)
AutoDD	**96.7 (1)**	**0.93 (1)**	**98.4 (1)**	**0.93 (1)**	**82.4 (1)**	**0.90 (1)**	75.8 (3)	**0.84 (1)**	70.9 (4)	**0.88 (1)**	84.8 (2)	**0.90 (1)**

TABLE V

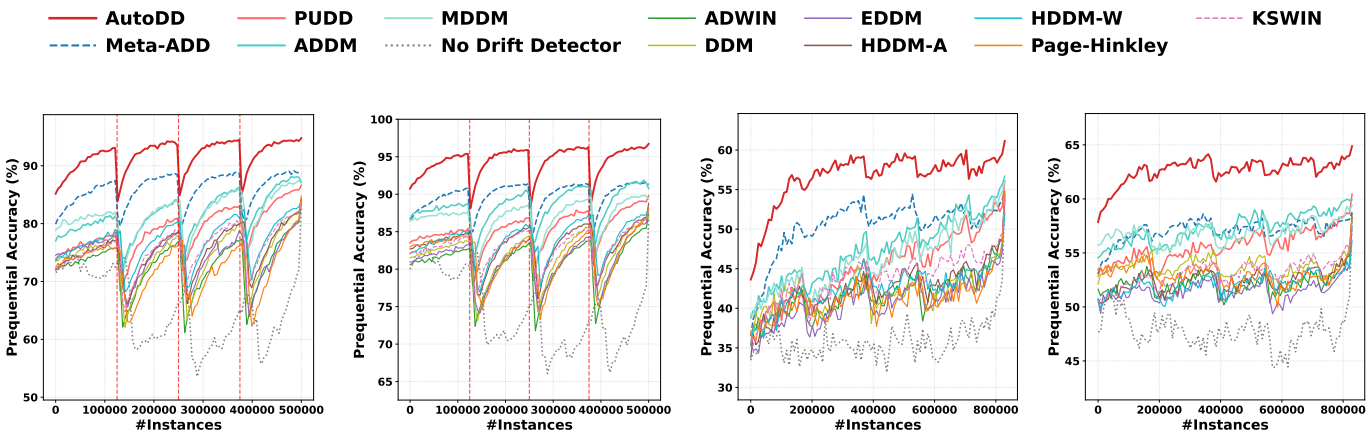
REAL-WORLD DATASETS UNDER GAUSSIANNB. RANK IN (·). **BOLD** = BEST, UNDERLINE = SECOND-BEST PER COLUMN. A LOWER DN RANK IS BETTER.

Method	Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
No Detector	73.2 (13)	**0 (1)**	68.9 (12)	**0 (1)**	77.2 (12)	**0 (1)**	59.3 (13)	**0 (1)**	54.6 (6)	**0 (1)**	66.6 (13)	**0 (1)**
ADWIN	79.2 (11)	22 (3)	69.3 (11)	7 (4)	92.1 (9)	2 (3)	62.5 (7)	76 (5)	54.3 (7)	3 (2)	71.5 (10)	22 (4)
DDM	82.3 (8)	22 (3)	70.6 (9)	5 (2)	91.8 (11)	2 (3)	61.8 (9)	20 (2)	54.2 (8)	3 (2)	72.1 (8)	10.4 (2)
EDDM	82.4 (7)	22 (3)	73.8 (6)	9 (5)	92.9 (5)	2 (3)	61.3 (11)	182 (11)	53.6 (10)	4 (3)	72.8 (7)	43.8 (6)
HDDM-A	82.7 (6)	22 (3)	72.2 (8)	9 (5)	92.5 (8)	2 (3)	62.6 (6)	114 (7)	54.6 (6)	6 (4)	72.9 (6)	30.6 (5)
HDDM-W	81.9 (9)	22 (3)	72.6 (7)	9 (5)	91.9 (10)	1 (2)	61.0 (12)	269 (12)	52.3 (11)	394 (12)	71.9 (9)	139 (13)
Page-Hinkley	78.0 (12)	16 (2)	69.4 (10)	6 (3)	91.8 (11)	1 (2)	62.8 (5)	64 (3)	54.3 (7)	17 (5)	71.3 (12)	20.8 (3)
KSWIN	81.1 (10)	22 (3)	68.0 (13)	9 (5)	92.6 (7)	2 (3)	61.6 (10)	269 (12)	53.7 (9)	266 (11)	71.4 (11)	113.6 (12)
MDDM	83.9 (5)	22 (3)	74.8 (5)	9 (5)	92.8 (6)	2 (3)	62.2 (8)	110 (6)	56.1 (4)	190 (10)	74.0 (5)	66.6 (9)
ADDM	84.8 (3)	22 (3)	76.3 (2)	9 (5)	93.9 (3)	2 (3)	63.9 (3)	74 (4)	56.5 (2)	177 (7)	75.1 (2)	56.8 (8)
PUDD	84.4 (4)	22 (3)	75.4 (3)	9 (5)	93.2 (4)	2 (3)	64.0 (2)	117 (8)	56.2 (3)	81 (6)	74.6 (4)	46.2 (7)
Meta-ADD	85.4 (2)	22 (3)	74.9 (4)	9 (5)	94.5 (2)	2 (3)	63.8 (4)	139 (10)	55.5 (5)	182 (8)	74.8 (3)	70.8 (11)
AutoDD	**89.6 (1)**	22 (3)	**80.0 (1)**	9 (5)	**97.9 (1)**	2 (3)	**68.2 (1)**	118 (9)	**59.7 (1)**	188 (9)	**79.1 (1)**	67.8 (10)

TABLE VI

REAL-WORLD DATASETS UNDER HAT. RANK IN (·). **BOLD** = BEST, UNDERLINE = SECOND-BEST PER COLUMN. A LOWER DN RANK IS BETTER.

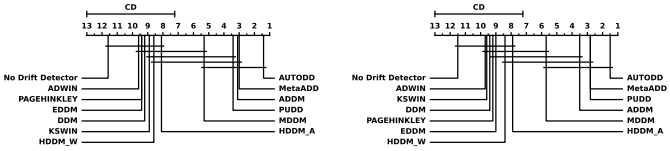
Method	Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
No Detector	76.5 (13)	**0 (1)**	71.4 (12)	**0 (1)**	80.1 (12)	**0 (1)**	62.1 (12)	**0 (1)**	56.9 (11)	**0 (1)**	69.4 (12)	**0 (1)**
ADWIN	82.4 (11)	22 (3)	73.1 (10)	9 (4)	94.6 (8)	2 (3)	66.3 (5)	76 (5)	57.7 (8)	3 (2)	74.8 (9)	22.4 (4)
DDM	85.5 (7)	22 (3)	73.5 (9)	7 (3)	93.9 (11)	2 (3)	65.2 (9)	20 (2)	58.0 (6)	3 (2)	75.2 (8)	10.8 (2)
EDDM	85.2 (8)	22 (3)	77.1 (6)	9 (4)	94.8 (6)	2 (3)	64.6 (11)	182 (11)	57.8 (7)	4 (3)	75.9 (6)	43.8 (6)
HDDM-A	86.3 (6)	22 (3)	75.8 (8)	9 (4)	94.7 (7)	2 (3)	66.1 (6)	114 (7)	58.5 (5)	6 (4)	76.3 (5)	30.6 (5)
HDDM-W	85.1 (9)	22 (3)	76.0 (7)	9 (4)	94.5 (9)	1 (2)	64.9 (10)	269 (12)	56.8 (12)	404 (12)	75.5 (7)	141 (13)
Page-Hinkley	81.7 (12)	16 (2)	72.8 (11)	6 (2)	94.2 (10)	1 (2)	66.5 (4)	64 (3)	57.5 (9)	17 (5)	74.5 (11)	20.8 (3)
KSWIN	84.3 (10)	22 (3)	71.2 (13)	9 (4)	94.9 (5)	2 (3)	65.4 (8)	269 (12)	57.0 (10)	266 (11)	74.6 (10)	113.6 (12)
MDDM	86.5 (5)	22 (3)	78.0 (5)	9 (4)	94.6 (8)	2 (3)	66.0 (7)	110 (6)	60.0 (2)	190 (10)	77.0 (4)	66.6 (9)
ADDM	87.3 (4)	22 (3)	79.2 (2)	9 (4)	95.8 (3)	2 (3)	66.9 (3)	74 (4)	59.6 (4)	177 (7)	77.8 (2)	56.8 (8)
PUDD	87.5 (3)	22 (3)	78.6 (3)	9 (4)	95.6 (4)	2 (3)	67.7 (2)	117 (8)	59.7 (3)	81 (6)	77.8 (2)	46.2 (7)
Meta-ADD	87.9 (2)	22 (3)	78.2 (4)	9 (4)	95.9 (2)	2 (3)	66.5 (4)	139 (10)	59.6 (4)	182 (8)	77.6 (3)	70.8 (11)
AutoDD	**93.8 (1)**	22 (3)	**85.1 (1)**	9 (4)	**99.0 (1)**	2 (3)	**72.6 (1)**	118 (9)	**65.1 (1)**	188 (9)	**83.1 (1)**	67.8 (10)



(a) SEA (GNB) (b) SEA (HAT) (c) Poker (GNB) (d) Poker (HAT)

Fig. 2. Prequential accuracy over time. SEA (synthetic) and Poker (real-world) datasets with GaussianNB and HAT base models.

the same clique. The consistent ranking across GaussianNB and HAT indicates that AutoDD's advantage is not tied to a particular downstream learner.



(a) GaussianNB (b) HAT

Fig. 3. Critical Difference diagrams (Friedman–Nemenyi, $\alpha = 0.05$). AutoDD achieves the best average rank under both base models.

D. Ablation Study

Table VII reports performance when each component is removed in turn, using GaussianNB averaged across the Elec, Weather, Airlines, and Poker streams. Removing the Level 1 LinUCB detector selector causes the largest accuracy drop, confirming that detector-stream matching is the primary driver of adaptive performance. Removing Level 2 Thompson Sampling produces a smaller but meaningful degradation, and all three meta-feature categories provide complementary gains, with statistical features contributing most. Figure 4 illustrates the resulting behaviour on the HYP stream. The LinUCB layer continuously re-selects detectors as meta-features evolve, keeping the windowed error bounded and enabling rapid recovery after detected drifts.

E. Sensitivity Study

Figure 5 shows AutoDD's sensitivity to its three key hyperparameters. Performance peaks at $w = 100$, where smaller windows introduce high variance and larger windows delay drift response. The exploration parameter α_{ucb}, fixed at 1.0 in all experiments rather than the horizon-dependent value of Theorem 1, shows a clear optimum, with under-exploration causing exploitation bias and over-exploration reducing accuracy. The regularisation parameter λ_{reg} is robust across the tested range, and AutoDD degrades gracefully around the default settings.

TABLE VII
ABLATION STUDY RESULTS (ACCURACY AND MACRO-F1 AVERAGED ACROSS REAL-WORLD DATASETS UNDER GAUSSIANNB).

Configuration	Accuracy	Macro-F1	ΔAcc
Full AutoDD	**74.5%**	**0.60**	–
w/o Temporal	67.9%	0.54	-6.5%
w/o Statistical	64.9%	0.52	-9.6%
w/o Stationarity	67.5%	0.54	-7.0%
w/o LinUCB (Level 1)	56.6%	0.45	-17.9%
w/o Thompson (Level 2)	69.2%	0.55	-5.3%

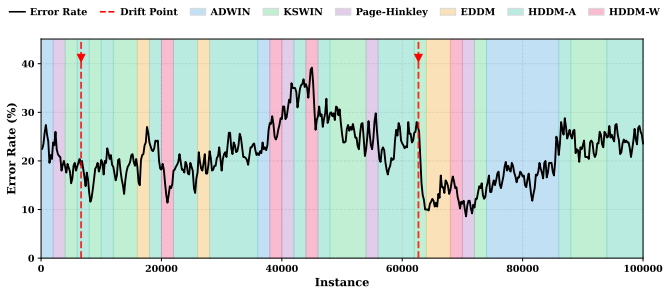
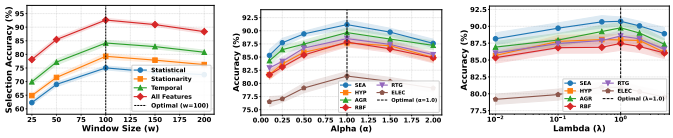


Fig. 4. Detector selection by AutoDD over the first 100,000 instances of the HYP stream. All quantitative results use the full 500,000-instance stream. Coloured bands show the dominant LinUCB-selected detector per block, the black curve the windowed prequential error rate, and dashed red lines the drift alarms raised by AutoDD, not ground-truth drift points.

F. Computational Efficiency

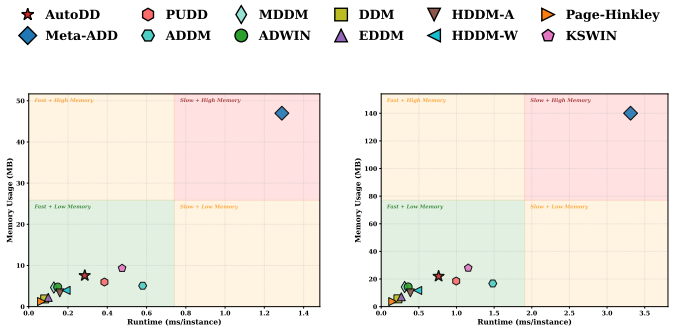
Figures 6a and 6b place all methods in the runtime-memory plane. AutoDD occupies the fast, low-memory region under both base models, with the hierarchical selection mechanism introducing only modest overhead. Most classical detectors also remain in this region, whereas Meta-ADD incurs substantially higher runtime and memory costs. AutoDD remains close to the lightweight fixed detectors despite performing



(a) w (b) α_{ucb} (c) λ_{reg}

Fig. 5. Hyperparameter sensitivity analysis showing prequential accuracy across different values of (a) window size w, (b) exploration parameter α_{ucb}, and (c) regularisation parameter λ_{reg}.

online detector and configuration selection. It is strictly more efficient than Meta-ADD and PUDD while attaining the better mean rank, making it practical for high-velocity streams.



(a) GaussianNB (b) HAT

Fig. 6. Runtime vs. memory trade-off. AutoDD lies in the Fast + Low Memory quadrant under both base models.

VII. Discussion and Limitations

We highlight several boundaries of the current design. AutoDD automates detector and hyperparameter selection but not the tumbling-window size, which the sensitivity analysis shows to be the most influential setting. The default value is robust across our benchmarks, yet selecting it for an unseen stream remains a manual step that an outer adaptive loop could close. The reward is an accuracy-based proxy and therefore entangles detector quality with the downstream learner. We mitigate this through window-level smoothing and the significance threshold, and empirical results confirm that selection preferences are consistent across base learners, though a fully label-free reward remains an open direction. To isolate detection from model selection we fix the base learner to GaussianNB and HAT, which are two standard but limited families, and adapt via full model reset, while richer learners and softer adaptation strategies (warm-starting, selective parameter resets, replay, ensemble updating) are left to future work. The comparison is also asymmetric in two respects that temper the reported margins. The four strongest comparators (MDDM, ADDM, PUDD, and Meta-ADD) are evaluated as baselines but are not members of the portfolio AutoDD selects from, so the action space is restricted to the classical detectors of Table I. In addition, all baselines run at fixed literature-recommended hyperparameters while AutoDD tunes its detector online, so part of its advantage is attributable to tuning rather than to selection. Separating these two effects requires a comparator granted an equivalent tuning budget, which we leave to future work. AutoDD targets tabular data streams, the prevalent setting in the drift-detection literature. Non-tabular modalities such as image, audio, and text are outside the scope of this work [35].

VIII. Conclusion and Future Work

This paper addressed a component that streaming AutoML has so far left to manual specification, namely the drift detector itself. We formalised the joint choice of detector and hyperparameters as a contextual selection problem and introduced AutoDD, which solves it online with a two-level hierarchical bandit. Level 1 uses LinUCB over 15 online meta-features to select a detector, and Level 2 uses Thompson Sampling to select its configuration, both driven by a bounded proxy reward because no drift labels are available at run time. Across ten benchmarks and two base learners, AutoDD attains the best mean accuracy and macro-F1 and the best average rank under the Friedman–Nemenyi test, while the ablation confirms detector selection as the main driver and its runtime and memory stay in the fast, low-memory region.

Several directions follow from these limitations. The most immediate is learning the window size w in an outer adaptive loop, since the sensitivity analysis identifies it as the most influential setting. A second is replacing the accuracy-based proxy with a label-free reward that decouples detector credit from the downstream learner. Finally, evaluating on streams with annotated change points would allow detection delay and false alarms to be scored directly.

References

[1] J. ao Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.

[2] A. Bifet, G. Holmes, B. Pfahringer, P. Kranen, H. Kremer, T. Jansen, and T. Seidl, "MOA: Massive online analysis, a framework for stream classification and clustering," in *Proceedings of the First Workshop on Applications of Pattern Analysis*. PMLR, 2010, pp. 44–50.

[3] V. Losing, B. Hammer, and H. Wersing, "Incremental on-line learning: A review and comparison of state of the art algorithms," *Neurocomputing*, vol. 275, pp. 1261–1274, 2018.

[4] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[5] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Brazilian Symposium on Artificial Intelligence*. Springer, 2004, pp. 286–295.

[6] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM International Conference on Data Mining*. SIAM, 2007, pp. 443–448.

[7] A. Pesaranghader, H. Viktor, and E. Paquet, "McDiarmid drift detection methods for evolving data streams," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2017, pp. 1663–1670.

[8] D. Kifer, S. Ben-David, and J. Gehrke, "Detecting change in data streams," in *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, vol. 4, 2004, pp. 180–191.

[9] C. Raab, M. Heusinger, and F.-M. Schleif, "Reactive soft prototype computing for concept drift streams," *Neurocomputing*, vol. 416, pp. 340–351, 2020.

[10] P. Lu, J. Lu, A. Liu, and G. Zhang, "Early concept drift detection via prediction uncertainty," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.

[11] H. Yu, J. Lu, and G. Zhang, "Meta-ADD: A meta-learning based pre-trained model for concept drift active detection," *Information Sciences*, vol. 578, pp. 1–14, 2021.
[12] V. M. A. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. A. P. A. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1805–1858, 2020.
[13] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *Fourth International Workshop on Knowledge Discovery from Data Streams*, vol. 6, 2006, pp. 77–86.
[14] F. Hutter, L. Kotthoff, and J. Vanschoren, "Automated machine learning: Methods, systems, challenges," in *Automated Machine Learning*. Springer, 2019.
[15] B. Celik, P. Singh, and J. Vanschoren, "Online AutoML: An adaptive AutoML framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.
[16] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "ASML: A scalable and efficient AutoML solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning (AutoML)*, ser. Proceedings of Machine Learning Research. PMLR, 2024.
[17] R. S. M. d. Barros and S. G. T. d. C. d. Santos, "A large-scale comparison of concept drift detectors," *Information Sciences*, vol. 451–452, pp. 348–370, 2018.
[18] C. Alippi, G. Boracchi, and M. Roveri, "Hierarchical change-detection tests," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 2, pp. 246–258, 2017.
[19] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th International Conference on World Wide Web (WWW)*. ACM, 2010, pp. 661–670.
[20] S. Agrawal and N. Goyal, "Analysis of Thompson sampling for the multi-armed bandit problem," in *Proceedings of the 25th Annual Conference on Learning Theory (COLT)*. PMLR, 2012, pp. 39.1–39.26.
[21] I. Friás-Blanco, J. de Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 3, pp. 810–823, 2015.
[22] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
[23] M. Z. A. Mayaki and M. Riveill, "Autoregressive based drift detection method," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2022.
[24] R. S. Tsay, "Testing and modeling threshold autoregressive processes," *Journal of the American Statistical Association*, vol. 84, pp. 231–240, 1989. [Online]. Available: https://api.semanticscholar.org/CorpusID:17630406
[25] T. Dasu, S. Krishnan, S. Venkatasubramanian, and K. Yi, "An information-theoretic approach to detecting changes in multidimensional data streams," in *Proceedings of Symposium on the Interface of Statistics, Computing Science, and Applications*, 2006.
[26] N. Lu, G. Zhang, and J. Lu, "Concept drift detection via competence models," *Artificial Intelligence*, vol. 209, pp. 11–28, 2014.
[27] D. M. dos Reis, P. Flach, S. Matwin, and G. Batista, "Fast unsupervised online drift detection using incremental Kolmogorov-Smirnov test," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1545–1554.
[28] C. Kulbach, J. Montiel, M. Bahri, M. Heyden, and A. Bifet, "Evolution-based online automated machine learning," in *Advances in Knowledge Discovery and Data Mining (PAKDD)*, ser. Lecture Notes in Computer Science, vol. 13280. Springer, Cham, 2022, pp. 472–484.
[29] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 2312–2320.
[30] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.
[31] V. Dani, T. P. Hayes, and S. M. Kakade, "Stochastic linear optimization under bandit feedback," in *Proceedings of the 21st Annual Conference on Learning Theory*, 2008, pp. 355–366.
[32] S. Agrawal and N. Goyal, "Further optimal regret bounds for Thompson sampling," in *Proceedings of the 16th International Conference on Artificial Intelligence and Statistics*. PMLR, 2013, pp. 99–107.
[33] A. Bifet and R. Gavaldà, "Adaptive learning from evolving data streams," in *Advances in Intelligent Data Analysis VIII (IDA 2009)*, ser. LNCS, vol. 5772. Springer, 2009, pp. 249–260.
[34] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, "River: Machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.
[35] V. M. A. Souza, A. R. S. Parmezan, F. A. Chowdhury, and A. Mueen, "Efficient unsupervised drift detector for fast and high-dimensional data streams," *Knowledge and Information Systems*, vol. 63, no. 6, pp. 1497–1527, 2021.

APPENDIX

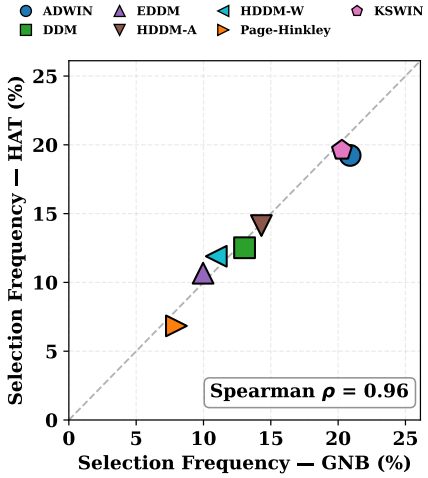


Fig. 7. Mean detector selection frequency under GaussianNB against HAT across all ten datasets. Every detector lies near the diagonal (Spearman $\rho = 0.96$, $p < 0.001$), indicating that the proxy reward tracks detector quality rather than base-learner artefacts.

TABLE VIII
ACCURACY (%) BROKEN DOWN BY DRIFT TYPE UNDER GAUSSIANNB. AUTODD LEADS ON SUDDEN, INCREMENTAL, AND RECURRING DRIFT, WHILE META-ADD EDGES AHEAD ON GRADUAL DRIFT OWING TO ITS LEARNED DRIFT-TYPE REPRESENTATIONS.

Method	Sudden	Gradual	Incremental	Recurring
ADWIN	82.4	71.3	68.5	75.2
DDM	79.8	74.6	72.1	71.8
EDDM	75.2	78.4	74.8	69.3
HDDM-A	81.7	72.1	69.8	74.5
HDDM-W	80.1	76.2	73.5	74.6
Page-Hinkley	81.9	70.8	68.2	75.8
KSWIN	76.5	77.8	75.2	72.1
MDDM	81.3	75.8	74.1	73.9
ADDM	82.9	77.2	75.6	74.8
PUDD	82.1	79.5	76.8	75.4
Meta-ADD	84.6	**82.3**	78.9	77.8
AutoDD	**88.2**	81.7	**80.4**	**79.6**

TABLE IX

DETECTOR SELECTION FREQUENCY (%) BY STREAM CHARACTERISTIC. THE LEARNED POLICY ALIGNS WITH DOMAIN KNOWLEDGE, FAVOURING ADWIN AND DDM ON HIGH-VELOCITY STREAMS AND KSWIN ON HIGH-NOISE AND HIGH-DIMENSIONAL STREAMS.

Detector	High Velocity	High Noise	High Dim.
ADWIN	28.4	12.3	15.7
DDM	22.1	8.5	11.2
EDDM	8.3	18.7	14.5
HDDM-A	15.2	10.4	12.8
HDDM-W	12.6	14.2	13.1
Page-Hinkley	5.8	8.1	7.2
KSWIN	7.6	27.8	25.5

TABLE X

MEAN DETECTION DELAY (INSTANCES) ON THE SYNTHETIC DATASETS WITH DISCRETE CHANGE POINTS, LOWER IS BETTER. AUTODD ATTAINS THE LOWEST DELAY OF ALL COMPARATORS, SELECTING RESPONSIVE DETECTORS DURING DRIFT-PRONE WINDOWS WHILE AVOIDING OVER-SENSITIVE CHOICES IN STABLE PERIODS.

Method	SEA	HYP	AGR	RBF	Mean
ADWIN	45.2	78.3	52.1	81.4	64.3
DDM	38.7	62.5	48.9	65.2	53.8
EDDM	52.3	71.8	61.4	74.6	65.0
HDDM-A	43.8	75.1	50.3	78.6	62.0
HDDM-W	41.5	58.2	45.7	62.8	52.1
Page-Hinkley	46.9	80.5	53.8	83.2	66.1
KSWIN	49.1	68.7	58.2	71.4	61.9
MDDM	39.4	55.8	44.2	60.5	50.0
ADDM	36.5	51.3	41.6	57.8	46.8
PUDD	32.8	45.2	37.5	51.6	41.8
Meta-ADD	35.2	48.6	39.8	55.3	44.7
AutoDD	**28.4**	**42.1**	**34.2**	**48.7**	**38.4**

TABLE XI

AUTODD PROXY-ORACLE SELECTION AGREEMENT (%) BY STREAM QUARTER. AGREEMENT RISES MONOTONICALLY FROM THE FIRST TO THE FINAL QUARTER ON EACH DATASET SHOWN, SHOWING THAT THE LINUCB LAYER LEARNS RELIABLE DETECTOR SELECTIONS FROM THE REWARD SIGNAL AS THE STREAM PROGRESSES.

Dataset	0–25%	25–50%	50–75%	75–100%
SEA	71.2	85.4	91.8	94.3
HYP	68.5	82.1	89.6	93.1
AGR	65.8	79.3	87.2	91.5
Elec	72.4	84.7	90.3	92.8
Poker	69.1	81.6	88.9	93.6
Mean	69.4	82.6	89.6	93.1

TABLE XII

AUTODD HYPERPARAMETER SETTINGS USED IN ALL EXPERIMENTS, FOR REPRODUCIBILITY. DETECTOR-SPECIFIC HYPERPARAMETER RANGES AND DEFAULTS ARE GIVEN IN TABLE I.

Component	Parameter	Value
Stream	Window size w	100
	Meta-feature dimensionality m	15
Portfolio	Number of detectors K	7
	Configurations per detector C	5
Level 1 (LinUCB)	Exploration α_{ucb}	1.0
	Regularisation λ_{reg}	1.0
Level 2 (Thompson)	Beta prior (a_0, b_0)	$(1, 1)$
Reward (Eq. (10))	Significance threshold τ_{sig}	0.05
	True-positive base ρ_{tp}	0.5
	Accuracy-drop scale κ	2
	False-positive penalty ρ_{fp}	0.3
	False-negative base ρ_{fn}	0.5
	True-negative reward ρ_{tn}	0.1
Evaluation	Runs per dataset	10
	Random seeds	$\sim \mathcal{U}\{1, \ldots, 100\}$

TABLE XIII

THE 15 ONLINE META-FEATURES FORMING THE CONTEXT VECTOR $\boldsymbol{\phi}_t$, GROUPED BY CATEGORY, WITH THE CLOSED-FORM EXPRESSION OR ONLINE ESTIMATOR USED TO COMPUTE EACH OVER THE TUMBLING WINDOW OF SIZE w.

Category	Feature	Formula/Description
Statistical	Velocity	$w/\Delta t_w$ (Δt_w = window elapsed time)
	Dimensionality	d_{in}
	Sparsity	$\frac{1}{w d_{\text{in}}} \sum_{i=1}^{w} \sum_{j=1}^{d_{\text{in}}} \mathbb{I}[\mathbf{x}_{i,j} = 0]$
	Noise level	$\frac{1}{d_{\text{in}}} \sum_{j=1}^{d_{\text{in}}} \sigma_j / \text{range}_j$
	Class imbalance	$\max_y n_y / \min_y n_y$ (n_y = label count)
Stationarity	Error volatility	$\sigma_{\text{err}}(t) = \sqrt{\frac{1}{w} \sum_{i=t-w+1}^{t} (e_i - \bar{e})^2}$
	KL divergence	$D_{\text{KL}}(P_t \Vert P_{t-1})$ ($\sqrt{w}$-bin histogram)
	Autocorrelation	$\rho_t = \frac{\text{Cov}(e_t, e_{t-1})}{\sigma_{e_t} \sigma_{e_{t-1}}}$
	Mean shift	$\Delta\mu_t = \frac{1}{d_{\text{in}}} \sum_{j=1}^{d_{\text{in}}} \lvert\mu_{t,j} - \mu_{t-1,j}\rvert$
	Variance shift	$\Delta\sigma_t^2 = \frac{1}{d_{\text{in}}} \sum_{j=1}^{d_{\text{in}}} \lvert\sigma_{t,j}^2 - \sigma_{t-1,j}^2\rvert$
Temporal	Trend	$\beta_{\text{trend}} = \frac{\text{Cov}(t, e_t)}{\text{Var}(t)}$
	Periodicity	$f_{\text{dom}} = \arg\max_f \lvert\mathcal{F}\{e_t\}(f)\rvert$
	Change frequency	$\frac{1}{w} \sum_{i=2}^{w} \mathbb{I}[\lvert\Delta e_i\rvert > \tau_{\Delta e}]$, $\tau_{\Delta e} = \sigma_e/2$
	Change magnitude	$\frac{1}{\lvert\mathcal{C}_t\rvert} \sum_{i \in \mathcal{C}_t} \lvert\Delta e_i\rvert$
	Recurrence score	$\text{corr}(e_{1:w/2},\, e_{w/2+1:w})$